Docker Complete Course - Levels 5 to 15

Deep Dive into Image Management, Container Operations, and Docker Compose

LEVEL 5: Docker Image Deep Learning

Topic 1: Deep Understanding of Images

Image Architecture:

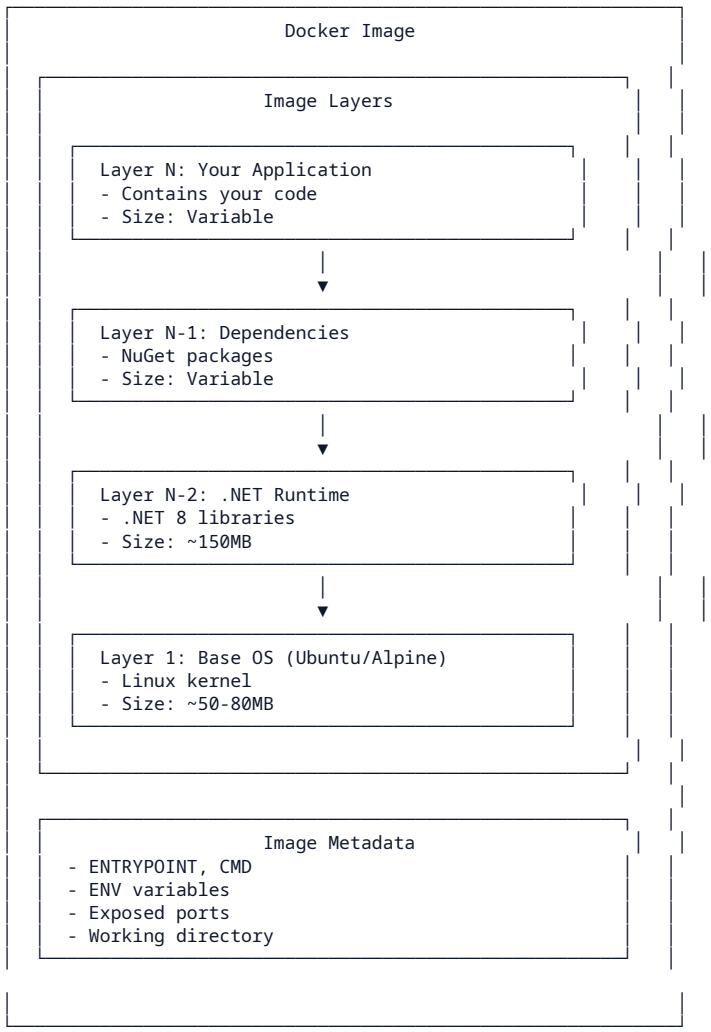


Image Storage Location:

```
# Windows
C:\ProgramData\docker\windowsfilter\

# Linux
/var/lib/docker/image/

# Mac
~/Library/Containers/com.docker.docker/Data/vms/0/
```

Topic 2: Base Image

Definition: Base image root layer hai jo Docker image build karne ke liye starting point provide karta hai. Yeh usually OS ya programming language runtime hota hai.

Base Image Examples:

```
# .NET Base Images
FROM mcr.microsoft.com/dotnet/aspnet:8.0        # .NET 8 runtime
FROM mcr.microsoft.com/dotnet/sdk:8.0           # .NET 8 SDK (for building)
FROM mcr.microsoft.com/dotnet/aspnet:6.0        # .NET 6 runtime
FROM mcr.microsoft.com/dotnet/sdk:6.0           # .NET 6 SDK

# OS Base Images
```

```
FROM ubuntu:22.04                           # Ubuntu Linux
FROM alpine:3.19                            # Minimal Alpine Linux
FROM debian:bookworm                        # Debian Linux
FROM windowsservercore:ltsc2022             # Windows Server Core

# Language Runtime
FROM node:20-alpine                         # Node.js 20
FROM python:3.12-slim                       # Python 3.12
FROM openjdk:17-slim                        # Java 17
FROM ruby:3.3-alpine                        # Ruby 3.3
```

Alpine vs Ubuntu:

```
Alpine:
- Very small (~5MB base)
- Uses musl libc (not glibc)
- May have compatibility issues with some software
- Good for production (small size)

Ubuntu:
- Larger (~80MB base)
- Uses glibc (widely compatible)
- More packages available
- Good for development (compatibility)
```

For .NET Developer:

```
# Development - Can use larger image for tools
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
RUN dotnet build

# Production - Use smaller runtime image
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
# Only runtime, no build tools
```

Topic 3: Official Image

Definition: Official images Docker Hub par verify kiye gaye images hain jo Docker maintain karta hai. Yeh security audited aur optimized hain.

Official Image Characteristics:

```
 Docker Verified Publisher
 Regular security updates
 Clear documentation
 Good practices followed
 Community supported
```

Official Image Examples:

```
# .NET Official Images
mcr.microsoft.com/dotnet/aspnet:8.0
mcr.microsoft.com/dotnet/sdk:8.0
mcr.microsoft.com/mssql/server:2022-latest

# Other Official Images
nginx:latest
redis:7-alpine
postgres:16
mysql:8.0
node:20-alpine
python:3.12-slim
```

How to Identify Official Images:

```
Docker Hub website:
- Shows "OFFICIAL IMAGE" badge
- Verified publisher badge

Docker search:
docker search nginx
# Output shows OFFICIAL column
# NAME      DESCRIPTION          STARS   OFFICIAL
# nginx     ...                  15000   [OK]
# bitnami/nginx ...              500     [ ]
```

Using Official Images:

```
# Safe to use for production
```

```
docker pull mcr.microsoft.com/dotnet/aspnet:8.0

# Always specify version, not just :latest
docker pull mcr.microsoft.com/dotnet/aspnet:8.0.0
```

Topic 4: Custom Image

Definition: Custom image aap khud banate ho apne application ke liye. Yeh official image par base hota hai aur aapke code aur dependencies contain karta hai.

Creating Custom Image:

```
# Step 1: Start from base image
FROM mcr.microsoft.com/dotnet/aspnet:8.0

# Step 2: Set working directory
WORKDIR /app

# Step 3: Copy application files
COPY . /app

# Step 4: Set environment
ENV ASPNETCORE_ENVIRONMENT=Production

# Step 5: Set entry point
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Build Custom Image:

```
# Build with default tag
docker build -t myapp .

# Build with version
docker build -t myapp:v1.0.0 .

# Build with registry prefix
docker build -t myregistry.azurecr.io/myapp:v1.0.0 .
```

Push to Registry:

```
# Tag for registry
docker tag myapp:v1.0.0 myusername/myapp:v1.0.0

# Push to Docker Hub
docker push myusername/myapp:v1.0.0

# Push to private registry
docker push myregistry.azurecr.io/myapp:v1.0.0
```

Topic 5: Image Caching

Definition: Docker build process mein layers cache karta hai. Agar layer unchanged hai, toh Docker cached version use karta hai jo build ko fast banata hai.

How Caching Works:

```
# Dockerfile
FROM ubuntu:22.04                    # Layer 1: Cached
WORKDIR /app                         # Layer 2: Cached
COPY package.json .                  # Layer 3: Cached (if unchanged)
RUN npm install                      # Layer 4: Cached (if unchanged)
COPY . .                             # Layer 5: Rebuilt (changed!)
```

Cache Optimization:

```
#  Good - Copy deps first, then code
FROM node:20
WORKDIR /app

# Copy package files first (rarely change)
COPY package*.json ./
RUN npm ci

# Copy rest of code (changes often)
COPY . .
```

```
RUN npm run build

#  Good - Use specific versions
FROM node:20.11.0  # Not just "20"

#  Bad - Copy everything at once
FROM node:20
COPY . .            # Any code change rebuilds everything
RUN npm install
```

Cache Busting:

```
# Force no-cache for specific stage
docker build --no-cache .

# Clear build cache
docker builder prune -a
```

Topic 6: Image Digest

Definition: Image digest SHA256 hash hai jo image content uniquely identify karta hai. Yeh tag se alag hai kyunki tag change ho sakta hai, digest nahi.

Why Digest Matters:

```
Tag can change:
- nginx:latest → New version might be pushed
- You pulled yesterday → Different from today?

Digest is immutable:
- sha256:abc123... → Always same content
- Can't be changed without new digest
```

View Digest:

```
# Pull and view digest
docker pull nginx@sha256:abc123...

# Inspect image digest
docker inspect --format='{{index .RepoDigests 0}}' nginx:latest

# Output:
# nginx@sha256:abc123def456...
```

Using Digest for Reproducibility:

```
# Instead of tag, use digest for exact image
FROM nginx@sha256:abc123def456...

# This guarantees exact same image everywhere
# No surprise updates!
```

Topic 7: Save and Load Images

Save Image to File:

```
# Save image to tar file
docker save -o myapp.tar myapp:v1.0.0

# Save multiple images
docker save -o images.tar image1:v1 image2:v1

# Compress while saving
docker save myapp:v1 | gzip > myapp.tar.gz
```

Load Image from File:

```
# Load from tar file
docker load -i myapp.tar

# Load from compressed file
docker load -i myapp.tar.gz

# Or with pipe
gunzip -c myapp.tar.gz | docker load
```

Use Cases:

```
# Share image without registry
# Machine A: Save and copy to USB
docker save myapp:v1 -o myapp.tar
# Copy myapp.tar to another machine

# Machine B: Load from USB
docker load -i myapp.tar

# Air-gapped environment (no internet)
# Build on development machine, transfer to production
```

Topic 8: Push Image to Registry

Push to Docker Hub:

```
# 1. Login to Docker Hub
docker login

# 2. Tag image with username
docker tag myapp:v1.0.0 yourusername/myapp:v1.0.0

# 3. Push image
docker push yourusername/myapp:v1.0.0

# 4. Also push :latest
docker tag myapp:v1.0.0 yourusername/myapp:latest
docker push yourusername/myapp:latest
```

Push to AWS ECR:

```
# 1. Get ECR login password
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin \
123456789012.dkr.ecr.us-east-1.amazonaws.com

# 2. Tag image
docker tag myapp:v1.0.0 \
123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0.0

# 3. Push image
docker push \
123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0.0
```

Push to Azure Container Registry:

```
# 1. Login
docker login myregistry.azurecr.io

# 2. Tag
docker tag myapp:v1.0.0 myregistry.azurecr.io/myapp:v1.0.0

# 3. Push
docker push myregistry.azurecr.io/myapp:v1.0.0
```

Topic 9: Image Security Scanning

Why Scan:

```
Vulnerabilities in images can:
- Expose sensitive data
- Allow unauthorized access
- Compromise entire system
- Lead to data breaches

Regular scanning prevents these!
```

Scan with Docker Scout:

```
# Enable Docker Scout (Docker Desktop Pro+)
docker scout cves myapp:v1

# View vulnerabilities
docker scout recommendations myapp:v1
```

Scan with Trivy:

```
# Install Trivy
docker run aquasec/trivy --version

# Scan image
docker run aquasec/trivy image myapp:v1

# Scan with output
docker run aquasec/trivy image --format json myapp:v1 > scan.json
```

Security Best Practices:

```
#  Use specific versions
FROM node:20.11.0  # Not just "20"

#  Use minimal images
FROM node:20-alpine  # Smaller = fewer vulnerabilities

#  Don't run as root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

#  No secrets in image
#  ENV API_KEY=secret  # Don't do this!
#  Use secrets at runtime: docker run -e API_KEY=$API_KEY
```

Topic 10: Private vs Public Images

Public Images:

```
 Free to pull
 Anyone can use
 Good for open source
 Security risk (anyone can push)
 Not for proprietary apps
```

Private Images:

```
 Only authorized users can pull
 Good for proprietary code
 Secure for business apps
 Requires authentication
 May have storage costs
```

Pull Private Image:

```
# Login first
docker login

# Then pull
docker pull myregistry.azurecr.io/private-app:v1

# Or with credentials
docker pull username/private-app:v1
```

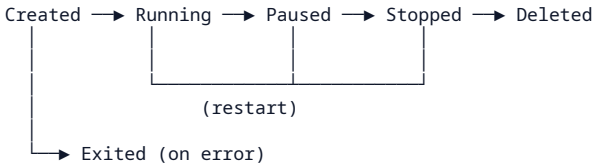
Best Practice:

```
# Public base images: Fine
docker pull nginx:latest
docker pull mcr.microsoft.com/dotnet/aspnet:8.0

# Private application images: Use private registry
# AWS ECR, Azure ACR, Google Artifact Registry
```

LEVEL 6: Docker Container Deep Learning

Topic 1: Container Lifecycle States

State Diagram:



State Details:

```
# Created: Container exists but not started
docker create nginx
# Status: Created

# Running: Container is executing
docker start mycontainer
# Status: Up 5 minutes

# Paused: Container processes suspended
docker pause mycontainer
# Status: Paused

# Stopped: Container exited gracefully
docker stop mycontainer
# Status: Exited (0)

# Killed: Container terminated forcefully
docker kill mycontainer
# Status: Exited (137)

# Restarting: Container is restarting
docker restart mycontainer
# Status: Restarting
```

Exit Codes:

```
Exit 0:     Successful exit, no errors
Exit 1:     Application error (general)
Exit 137:   SIGKILL (usually OOM - Out of Memory)
Exit 139:   SIGSEGV (Segmentation fault)
Exit 143:   SIGTERM (Graceful shutdown)
```

Topic 2: Container Modes

Detached Mode (-d):

```
# Run in background
docker run -d --name mynginx nginx

# Check it's running
docker ps
# CONTAINER ID   IMAGE   STATUS      PORTS     NAMES
# abc123         nginx   Up 5 secs             mynginx

# Container continues running after terminal closes
```

Interactive Mode (-it):

```
# Interactive with terminal
docker run -it ubuntu bash
# root@abc123:/#

# Run command and exit
docker run -it ubuntu echo "Hello"
# Output: Hello

# -i: Keep STDIN open
# -t: Allocate pseudo-TTY
```

Combined Modes:

```
# Detached + Interactive (for debugging)
docker run -d -it --name debug-container ubuntu bash

# Attach later
docker attach debug-container

# Exit without stopping: Ctrl+P, Ctrl+Q
```

One-off Commands:

```
# Run command in existing container
docker exec mycontainer ls /app

# Interactive shell
docker exec -it mycontainer bash

# Run and remove (one-off task)
docker run --rm alpine echo "Done"
```

Topic 3: Container Restart Policies

Why Restart Policy:

```
Without policy:
- Container crashes → stays stopped
- No automatic recovery
- Manual intervention needed

With restart policy:
- Container crashes → automatically restarts
- Self-healing
- Always running services
```

Available Policies:

```
# no - Never restart (default)
docker run -d --restart no nginx

# always - Always restart
docker run -d --restart always nginx

# on-failure - Restart only on error (non-zero exit)
docker run -d --restart on-failure nginx

# on-failure:3 - Restart on error, max 3 retries
docker run -d --restart on-failure:3 nginx

# unless-stopped - Restart unless manually stopped
docker run -d --restart unless-stopped nginx
```

Restart Policy Comparison:

```
# Use in docker-compose.yml
services:
  api:
    restart: "no"              # Don't restart
    # Use: Batch jobs, one-time tasks

  web:
    restart: "always"          # Always restart
    # Use: Web servers, APIs

  worker:
    restart: "on-failure"      # Restart on error
    # Use: Background workers, jobs

  critical:
    restart: "unless-stopped"  # Restart unless stopped
    # Use: Databases, critical services
```

Topic 4: Container Resource Limits

CPU Limits:

```
# Limit to 1 CPU core
docker run -d --cpus="1.0" myapi

# Limit to 0.5 CPU (half core)
```

```
docker run -d --cpus="0.5" myapi

# Limit specific cores
docker run -d --cpuset-cpus="0,1" myapi
```

Memory Limits:

```
# Limit to 512 MB
docker run -d --memory="512m" myapi

# Limit to 1 GB
docker run -d --memory="1g" myapi

# Limit with swap
docker run -d --memory="512m" --memory-swap="1g" myapi

# Memory + CPU
docker run -d \
  --memory="1g" \
  --cpus="2" \
  myapi
```

View Resource Usage:

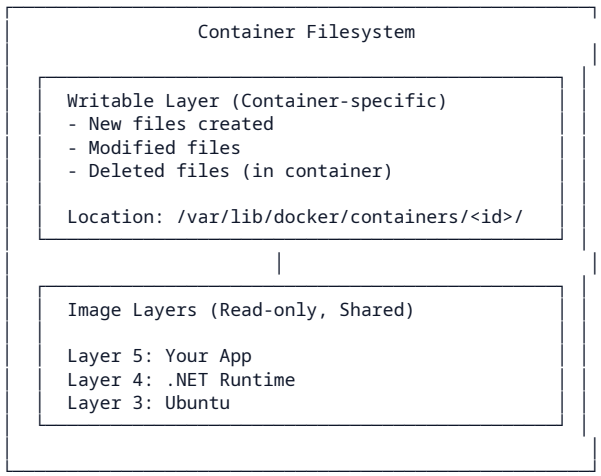
```
# Real-time stats
docker stats

# Stats for specific container
docker stats myapi

# All containers stats
docker stats $(docker ps --format '{{.Names}}')

# Inspect limits
docker inspect --format='{{json .HostConfig.Memory}}' myapi
```

docker-compose.yml Resources:

```
services:
  api:
    image: myapi
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 1G
        reservations:
          cpus: '0.5'
          memory: 512M
```

Topic 5: Container Filesystem

Filesystem Layers:



Copy Files To/From Container:

```
# Copy from host to container
docker cp ./config.json myapi:/app/config.json
```

```
# Copy from container to host
docker cp myapi:/app/logs ./logs

# Copy entire folder
docker cp myapi:/app/. ./local-app
```

Inspect Filesystem:

```
# Enter container and explore
docker exec -it myapi bash

# Check directory structure
ls -la /app

# Check disk usage
df -h

# Check specific folder size
du -sh /app
```

Topic 6: Container Health Checks

Why Health Check:

```
Without health check:
- Container running even if app crashed
- Load balancer sends traffic to dead container
- No automatic recovery

With health check:
- Docker tests if app is healthy
- Unhealthy containers marked
- Can auto-restart or remove from load balancer
```

Dockerfile HEALTHCHECK:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY . .

# Health check - test /health endpoint
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1

ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Add Health Endpoint in .NET:

```
// Program.cs
app.MapGet("/health", () => Results.Ok(new
{
    status = "healthy",
    timestamp = DateTime.UtcNow
}));
```

docker-compose.yml Health Check:

```
services:
  api:
    build: .
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    ports:
      - "5000:5000"
```

Check Health Status:

```
# View health status
docker inspect --format='{{.State.Health.Status}}' myapi

# Possible values: starting, healthy, unhealthy
```

Topic 7: Container Hostname and DNS

Default Hostname:

```
# Container hostname defaults to container ID (short)
docker run -d --name myapi nginx
docker exec myapi hostname
# Output: abc123def456

# Custom hostname
docker run -d --name myapi --hostname myapi-host nginx
docker exec myapi hostname
# Output: myapi-host
```

Docker DNS:

```
# Containers can resolve each other by name
docker network create mynet
docker run -d --network mynet --name api myapi
docker run -d --network mynet --name db mssql

# From api container, access db by name
docker exec api ping db
# Pings successfully!

# Also works with aliases
docker run -d --network mynet --name api --network-alias api-svc myapi
# Can be accessed as both "api" and "api-svc"
```

DNS Resolution Order:

```
1. Container's own hostname
2. Network aliases
3. Container name (if on same network)
4. Built-in DNS server (127.0.0.11)
```

Topic 8: Container Isolation

Namespace Isolation:

```
PID Namespace:     Each container sees only its own processes
NET Namespace:     Each container has its own network stack
IPC Namespace:     Each container has its own IPC resources
MNT Namespace:     Each container has its own filesystem view
UTS Namespace:     Each container has its own hostname
USER Namespace:    Each container has its own user IDs
```

Process Isolation:

```
# Host sees container process
ps aux | grep nginx
# Shows nginx process running

# But inside container
docker exec mynginx ps aux
# Only shows processes inside container
# Host processes not visible
```

Network Isolation:

```
# Container has own network
docker exec mynginx ip addr
# Shows: lo, eth0 (container's network)

# Container can't see host network
docker exec mynginx ip link show docker0  # Won't work
```

Filesystem Isolation:

```
# Container can't see host files (unless bind mounted)
docker run -it ubuntu ls /host
# ls: cannot access '/host': No such file or directory
```

Topic 9: Container Communication

Same Network:

```
# Create network
```

```
docker network create mynet

# Run containers on same network
docker run -d --network mynet --name api myapi
docker run -d --network mynet --name db sqlserver

# From api to db - use container name as hostname
# Connection string: Server=db;Database=MyDb;...
# "db" resolves to db container's IP
```

Different Networks:

```
# Container on multiple networks
docker network connect mynet api
docker network disconnect mynet api

# Inspect networks
docker inspect myapi --format='{{json .NetworkSettings.Networks}}'
```

Ports vs Network:

```
Ports (docker run -p 8080:5000):
- Expose container port to host
- External users access via host:8080

Network:
- Container-to-container communication
- Containers on same network talk to each other
- No external access required
```

Topic 10: Ephemeral Nature of Container

What Gets Lost:

```
When container is deleted:
 Files created inside container
 Modified configuration
 Database data (without volume)
 Application logs
 Any runtime changes

Data in volumes:  Saved
Data in bind mounts:  Saved
```

What Persists:

```
 Images (stored separately)
 Named volumes
 Bind mount host directories
 Image layers (read-only)
```

Example - Data Loss:

```
# Create file in container
docker run -d --name test nginx
docker exec test touch /app/data.txt
docker exec test ls /app
# Shows: data.txt

# Delete container - DATA LOST!
docker rm -f test

# Create new container
docker run -d --name test2 nginx
docker exec test2 ls /app
# No data.txt - it's gone!
```

Solution - Use Volumes:

```
# Create volume
docker volume create mydata

# Use volume
docker run -d --name test -v mydata:/app nginx

# Create file in volume
docker exec test touch /app/data.txt

# Delete container - Data preserved!
docker rm -f test
```

```
# New container with same volume - Data there!
docker run -d --name test2 -v mydata:/app nginx
docker exec test2 ls /app
# Shows: data.txt
```

LEVEL 7: Dockerfile Basics

Complete Dockerfile Instructions

FROM Instruction

Purpose: Base image define karta hai.

```
# Official .NET image
FROM mcr.microsoft.com/dotnet/aspnet:8.0

# Official OS
FROM ubuntu:22.04

# With alias (for multi-stage)
FROM ubuntu:22.04 AS build
```

WORKDIR Instruction

Purpose: Working directory set karta hai.

```
# Set working directory
WORKDIR /app

# Creates directory if doesn't exist
WORKDIR /app/src  # Creates both /app and /app/src

# Can use environment variables
WORKDIR $APP_HOME
```

COPY Instruction

Purpose: Files copy karta hai from build context to image.

```
# Copy all files
COPY . /app

# Copy specific file
COPY appsettings.json /app/

# Copy with destination
COPY ./src /app/src

# Copy with ownership (Linux)
COPY --chown=appuser:appgroup . /app
```

ADD Instruction

Purpose: COPY se similar but URLs aur archives handle kar sakta hai.

```
# Add remote URL (use COPY instead when possible)
ADD https://example.com/file.zip /app/

# Extract tar file (automatically)
ADD myapp.tar.gz /app/

# COPY is preferred for regular files (faster, more predictable)
```

RUN Instruction

Purpose: Commands execute karta hai during build.

```
# Shell form
RUN echo "Hello"

# Exec form (preferred)
RUN ["echo", "Hello"]

# Chained commands (reduces layers)
RUN apt-get update && \
    apt-get install -y curl && \
    apt-get clean

# .NET specific
RUN dotnet restore
RUN dotnet build -c Release
```

CMD Instruction

Purpose: Default command jo container start hote waqt run hoga.

```
# Exec form (preferred)
CMD ["dotnet", "MyApp.dll"]

# With arguments
CMD ["dotnet", "run", "--configuration", "Release"]

# Shell form (not recommended)
CMD echo "Container started"
```

ENTRYPOINT Instruction

Purpose: Main executable define karta hai.

```
# Exec form
ENTRYPOINT ["dotnet", "MyApp.dll"]

# Shell form
ENTRYPOINT dotnet MyApp.dll
```

EXPOSE Instruction

Purpose: Documentation ke liye port define karta hai.

```
EXPOSE 5000
EXPOSE 80 443
EXPOSE 8080 8443
```

ENV Instruction

Purpose: Environment variable set karta hai.

```
# Single variable
ENV ASPNETCORE_ENVIRONMENT=Production

# Multiple variables
ENV APP_NAME=MyApp \
    APP_VERSION=1.0.0 \
    APP_ENV=Production

# Use in commands
RUN echo $APP_NAME
```

ARG Instruction

Purpose: Build-time variables (not available at runtime).

```
# Define argument
ARG VERSION=1.0

# Use in build
RUN echo "Building version $VERSION"

# Pass at build time
# docker build --build-arg VERSION=2.0 .
```

USER Instruction

Purpose: User set karta hai for subsequent commands.

```
# Create user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# Set user
USER appuser
```

VOLUME Instruction

Purpose: Volume mount point declare karta hai.

```
VOLUME ["/app/data"]
VOLUME ["/app/logs", "/app/config"]
```

HEALTHCHECK Instruction

Purpose: Container health check define karta hai.

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1
```

LABEL Instruction

Purpose: Metadata add karta hai.

```
LABEL version="1.0"
LABEL description="My application"
LABEL maintainer="email@example.com"
```

LEVEL 8: Dockerfile for .NET Developer

Complete .NET Dockerfiles

Dockerfile for .NET Console App

```
# ═══════════════════════════════════════════════════════════
# Stage 1: Build
# ═══════════════════════════════════════════════════════════
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

# Set working directory
WORKDIR /src

# Copy project file and restore dependencies
COPY ["ConsoleApp.csproj", "./"]
RUN dotnet restore

# Copy all source files
COPY . .

# Build the application
RUN dotnet build -c Release -o /app/build

# Publish the application
RUN dotnet publish -c Release -o /app/publish

# ═══════════════════════════════════════════════════════════
# Stage 2: Runtime
# ═══════════════════════════════════════════════════════════
FROM mcr.microsoft.com/dotnet/runtime:8.0 AS runtime

# Set working directory
WORKDIR /app

# Copy published output from build stage
COPY --from=build /app/publish .

# Non-root user for security
RUN groupadd -r appgroup && \
    useradd -r -g appgroup appuser && \
    chown -R appuser:appgroup /app

USER appuser

# Run the application
ENTRYPOINT ["dotnet", "ConsoleApp.dll"]
```

Dockerfile for ASP.NET Core Web API

```
# ═══════════════════════════════════════════════════════════
# Stage 1: Build
# ═══════════════════════════════════════════════════════════
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

# Copy and restore (order matters for caching!)
COPY ["MyApi.csproj", "./"]
RUN dotnet restore

# Copy source and build
COPY . .
RUN dotnet build -c Release --no-restore -o /app/build

# Publish
RUN dotnet publish -c Release --no-restore -o /app/publish

# ═══════════════════════════════════════════════════════════
# Stage 2: Runtime
# ═══════════════════════════════════════════════════════════
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime

WORKDIR /app

# Copy published files
COPY --from=build /app/publish .

# Environment variables
ENV ASPNETCORE_ENVIRONMENT=Production
```

```
ENV ASPNETCORE_URLS=http://+:5000

# Expose port
EXPOSE 5000

# Set ownership
RUN chown -R appuser:appgroup /app 2>/dev/null || true

# Run the application
ENTRYPOINT ["dotnet", "MyApi.dll"]
```

Dockerfile for ASP.NET Core MVC

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY *.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet build -c Release --no-restore -o /app/build

RUN dotnet publish -c Release --no-restore -o /app/publish

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

COPY --from=build /app/publish .

ENV ASPNETCORE_ENVIRONMENT=Production
ENV ASPNETCORE_URLS=http://+:80

EXPOSE 80

ENTRYPOINT ["dotnet", "MyMvcApp.dll"]
```

Dockerfile for Worker Service

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY *.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release --no-restore -o /app/publish

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

COPY --from=build /app/publish .

# No port exposed (background worker)
ENV DOTNET_RUNNING_IN_CONTAINER=true

ENTRYPOINT ["dotnet", "MyWorker.dll"]
```

Development Dockerfile

```
# For development with hot reload
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS development

WORKDIR /src

# Mount source for hot reload
COPY . .

# Expose ports
EXPOSE 5000 5001

# Run in development mode
CMD ["dotnet", "watch", "run"]
```

.dockerignore File

```
# Git
.git
.gitignore
```

```
# Build outputs - IMPORTANT!
bin/
obj/
out/
publish/

# IDE
.vs/
.vscode/
*.user
*.suo

# Logs
*.log
logs/

# Test
TestResults/
coverage/

# Docker
Dockerfile*
docker-compose*
.dockerignore

# OS
Thumbs.db
.DS_Store

# Secrets
*.key
*.pem
appsettings.*.json
```

LEVEL 9: Docker Port Mapping

Complete Port Mapping Guide

Port Mapping Syntax

```
# Basic syntax
-p host_port:container_port

# Examples
-p 8080:5000      # Host 8080 → Container 5000
-p 443:443        # Same port both sides
-p 5000:5000      # Same port both sides

# Multiple ports
-p 8080:5000 -p 443:5001

# Bind to specific host IP
-p 127.0.0.1:8080:5000

# UDP port
-p 53:53/udp
```

Common Ports Reference

```
# .NET Applications
-p 5000:5000      # HTTP (Kestrel)
-p 5001:5001      # HTTPS (Kestrel alternative)
-p 8080:5000      # Common HTTP mapping

# SQL Server
-p 1433:1433      # Default SQL Server port

# PostgreSQL
-p 5432:5432      # Default PostgreSQL port

# MySQL
-p 3306:3306      # Default MySQL port

# Redis
-p 6379:6379      # Default Redis port

# MongoDB
-p 27017:27017    # Default MongoDB port

# RabbitMQ
-p 5672:5672      # AMQP
-p 15672:15672    # Management UI

# Nginx
-p 80:80          # HTTP
-p 443:443        # HTTPS

# Elasticsearch
-p 9200:9200      # REST API
-p 9300:9300      # Transport
```

Port Mapping in Docker Compose

```
services:
  api:
    ports:
      - "8080:5000"        # Host 8080 → Container 5000
      - "5001:5001"        # HTTPS
      - "127.0.0.1:8081:5000"  # Bind to localhost only

  database:
    ports:
      - "1433:1433"

  redis:
    ports:
      - "6379:6379"
```

Port Conflicts

```
# Error: port is already in use
# Solution 1: Use different host port
docker run -d -p 8081:5000 myapi
```

```
# Solution 2: Find and stop conflicting process
netstat -ano | findstr :8080
# Kill the process using port 8080

# Solution 3: Check what's using port
docker ps --format '{{.Ports}}'
# Shows all port mappings

# View published ports
docker port mycontainer
```

Port Mapping Verification

```
# Check port mapping for a container
docker port myapi
# Output: 5000/tcp -> 0.0.0.0:8080

# List all port mappings
docker ps --format "table {{.Names}}\t{{.Ports}}"

# Test connectivity
curl http://localhost:8080/api/health

# Check if port is listening
docker exec myapi netstat -tlnp | grep 5000
```

LEVEL 10: Docker Volumes

Complete Volume Guide

Volume Types

Named Volume:

```
# Create
docker volume create mydata

# Use
docker run -d -v mydata:/app/data myapi

# All data in /app/data persists even after container delete
```

Bind Mount:

```
# Map host directory to container
docker run -d -v C:\Projects\MyApp:/app myapi

# Linux/Mac
docker run -d -v /home/user/projects:/app myapi

# Read-only bind mount
docker run -d -v C:\Projects\MyApp:/app:ro myapi
```

tmpfs Mount (memory only):

```
# Store sensitive data in memory (not on disk)
docker run -d --tmpfs /app/secrets:rw myapi

# With size limit
docker run -d --tmpfs /app/secrets:size=100m myapi
```

Volume Commands

```
# Create volume
docker volume create myvolume

# List volumes
docker volume ls

# Inspect volume
docker volume inspect myvolume

# Remove volume
docker volume rm myvolume

# Remove all unused volumes
docker volume prune

# Remove all volumes ( Data loss!)
docker volume prune -a
```

Volume with SQL Server

```
# Run SQL Server with volume
docker run -d \
  --name sqlserver \
  -e "ACCEPT_EULA=Y" \
  -e "SA_PASSWORD=YourStrong@Passw0rd" \
  -v sqldata:/var/opt/mssql \
  -p 1433:1433 \
  mcr.microsoft.com/mssql/server:2022-latest

# Data persists:
# /var/opt/mssql in container → sqldata volume
```

Volume with PostgreSQL

```
docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:16
```

Volume with MongoDB

```
docker run -d \
  --name mongodb \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=secret \
  -v mongodata:/data/db \
  -p 27017:27017 \
  mongo:7
```

Backup and Restore

```
# Backup volume
docker run --rm \
  -v sqldata:/data \
  -v $(pwd):/backup \
  alpine \
  tar czf /backup/backup.tar.gz -C /data .

# Restore volume
docker volume create sqldata
docker run --rm \
  -v sqldata:/data \
  -v $(pwd):/backup \
  alpine \
  tar xzf /backup/backup.tar.gz -C /data
```

Volume Permissions

```
# Common issue: Permission denied
# Solution 1: Create directory first
mkdir -p ./data
chmod 777 ./data

# Solution 2: Use named volume (Docker manages permissions)
docker run -v myvolume:/app/data myapi

# Solution 3: Run as root
docker run -u root -v $(pwd):/app myapi

# Solution 4: Fix in Dockerfile
RUN chmod -R 755 /app/data
```

LEVEL 11: Docker Networking

Complete Networking Guide

Network Types

Bridge Network (Default):

```
# Default bridge - all containers can communicate via IP
docker run -d --name api myapi
docker run -d --name db mysql

# API can reach DB via IP (but not by name easily)
docker exec api ping 172.17.0.2  # Works
docker exec api ping db          # Doesn't work (default bridge)
```

Custom Bridge Network:

```
# Create custom network
docker network create mynet

# Run containers on custom network
docker run -d --network mynet --name api myapi
docker run -d --network mynet --name db mysql

# Now containers can communicate by name!
docker exec api ping db          # Works!
docker exec db ping api          # Works!
```

Host Network:

```
# Container uses host network directly
docker run -d --network host --name api myapi

# No port mapping needed - container uses host ports directly
# Container app on port 5000 = host port 5000
```

None Network:

```
# No network access
docker run -d --network none --name isolated alpine

# Container cannot communicate with anything
docker exec isolated ping google.com  # Won't work
```

Network Commands

```
# List networks
docker network ls

# Create network
docker network create mynet

# Inspect network
docker network inspect mynet

# Connect container to network
docker network connect mynet mycontainer

# Disconnect container from network
docker network disconnect mynet mycontainer

# Remove network
docker network rm mynet

# Remove unused networks
docker network prune
```

Network in Docker Compose

```
version: '3.8'

services:
  api:
    build: .
    networks:
      - backend
    ports:
```

```
      - "8080:5000"

  db:
    image: mssql/server:2022
    networks:
      - backend
    volumes:
      - sqldata:/var/opt/mssql

  redis:
    image: redis:7
    networks:
      - backend

networks:
  backend:
    driver: bridge
```

Container-to-Container Communication

```
# Scenario: .NET API connecting to SQL Server

# Step 1: Create network
docker network create appnet

# Step 2: Run SQL Server on network
docker run -d \
  --network appnet \
  --name sqlserver \
  -e "ACCEPT_EULA=Y" \
  -e "SA_PASSWORD=YourStrong@Passw0rd" \
  mssql/server:2022-latest

# Step 3: Run API on same network
docker run -d \
  --network appnet \
  --name myapi \
  -e "ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyDb;User=sa;Password=You
    rStrong@Passw0rd;TrustServerCertificate=True" \
  -p 8080:5000 \
  myapi

# API connects to SQL Server using container name "sqlserver"
```

DNS and Service Discovery

```
# Docker DNS automatically resolves:
# - Container names
# - Network aliases
# - Container IDs (short and full)

# Examples:
docker exec api ping sqlserver     # By container name
docker exec api ping db            # By alias (if configured)

# Connection strings:
Server=sqlserver;Port=1433;...     # Container name works!
Server=mydb;...                    # Host name works!
```

LEVEL 12: SQL Server with Docker

Complete SQL Server Guide

Run SQL Server Container

```
# Basic SQL Server run
docker run -d \
  --name sqlserver \
  -e "ACCEPT_EULA=Y" \
  -e "SA_PASSWORD=YourStrong@Passw0rd" \
  -e "MSSQL_PID=Developer" \
  -p 1433:1433 \
  mcr.microsoft.com/mssql/server:2022-latest

# With volume for data persistence
docker run -d \
  --name sqlserver \
  -e "ACCEPT_EULA=Y" \
  -e "SA_PASSWORD=YourStrong@Passw0rd" \
  -e "MSSQL_PID=Developer" \
  -p 1433:1433 \
  -v sqldata:/var/opt/mssql \
  mcr.microsoft.com/mssql/server:2022-latest
```

Environment Variables

```
# ACCEPT_EULA - Required! Accept End User License Agreement
ACCEPT_EULA=Y

# SA_PASSWORD - System Administrator password
# Requirements: 8+ chars, uppercase, lowercase, number, special char
SA_PASSWORD=YourStrong@Passw0rd

# MSSQL_PID - Product ID
# Values: Developer (free), Express, Standard, Enterprise
MSSQL_PID=Developer

# Additional options
MSSQL_COLLATION=SQL_Latin1_General_CP1_CI_AS
MSSQL_AGENT_ENABLED=true
```

Wait for SQL Server to Start

```
# SQL Server takes 30-60 seconds to start
# Check logs until ready
docker logs -f sqlserver

# Look for: "SQL Server is now ready for client connections"

# Or wait with script
until docker exec -it sqlserver \
  /opt/mssql-tools/bin/sqlcmd \
  -S localhost -U sa -P "YourStrong@Passw0rd" \
  -Q "SELECT 1" > /dev/null 2>&1; do
  echo "Waiting for SQL Server..."
  sleep 5
done
echo "SQL Server is ready!"
```

Connect to SQL Server

From SQL Server Management Studio (SSMS):

```
Server Name: localhost,1433
Authentication: SQL Server Authentication
Login: sa
Password: YourStrong@Passw0rd
```

From Azure Data Studio:

```
Server: localhost,1433
Authentication: SQL Login
User: sa
Password: YourStrong@Passw0rd
```

From .NET Application:

```
// Connection string for Docker SQL Server
"Server=localhost;Database=MyDb;User
  Id=sa;Password=YourStrong@Passw0rd;TrustServerCertificate=True;"

// Or using container name (if on same network)
"Server=sqlserver;Database=MyDb;User
  Id=sa;Password=YourStrong@Passw0rd;TrustServerCertificate=True;"
```

Run SQL Commands

```
# Connect via sqlcmd
docker exec -it sqlserver /opt/mssql-tools/bin/sqlcmd \
  -S localhost -U sa -P "YourStrong@Passw0rd"

# List databases
SELECT name FROM sys.databases;
GO

# Create database
CREATE DATABASE MyAppDB;
GO

# Use database
USE MyAppDB;
GO

# Create table
CREATE TABLE Products (
    Id INT PRIMARY KEY,
    Name NVARCHAR(100),
    Price DECIMAL(18,2)
);
GO

# Insert data
INSERT INTO Products VALUES (1, 'Laptop', 999.99);
GO

# Select data
SELECT * FROM Products;
GO

# Exit
EXIT
```

Docker Compose with SQL Server

```
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8080:5000"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyAppDB;User=sa;Passwor
        d=YourStrong@Passw0rd;TrustServerCertificate=True;
    depends_on:
      - sqlserver
    networks:
      - appnet

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
      - MSSQL_PID=Developer
    ports:
      - "1433:1433"
    volumes:
      - sqldata:/var/opt/mssql
    networks:
      - appnet

networks:
  appnet:
    driver: bridge

volumes:
  sqldata:
```

Health Check for SQL Server

```
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P
        'YourStrong@Passw0rd' -Q 'SELECT 1'"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s
```

LEVEL 13: Docker Compose Basics

Complete Docker Compose Guide

docker-compose.yml Structure

```
version: '3.8'  # Docker Compose file format version

services:  # Define all containers
  servicename:
    image: imagename         # Image to use
    build: ./path            # OR Build from Dockerfile
    container_name: myname   # Custom container name
    ports:                   # Port mappings
      - "8080:5000"
    environment:             # Environment variables
      - KEY=value
    volumes:                 # Volume mounts
      - myvol:/app/data
    networks:                # Network configuration
      - mynet
    depends_on:              # Dependency
      - database
    restart: always          # Restart policy
    healthcheck:             # Health check
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]

networks:  # Define networks
  mynet:
    driver: bridge

volumes:  # Define volumes
  myvol:
```

Basic Commands

```
# Start all services
docker compose up

# Start in detached mode
docker compose up -d

# Start with rebuild
docker compose up --build

# Stop all services
docker compose down

# Stop and remove volumes
docker compose down -v

# Stop and remove images
docker compose down --rmi all

# View running services
docker compose ps

# View logs
docker compose logs

# Follow logs
docker compose logs -f

# Execute command in service
docker compose exec api bash

# Run specific command
docker compose exec api dotnet ef database update

# Build images
docker compose build

# Restart services
docker compose restart

# Scale service
docker compose up -d --scale api=3

# Show running processes
docker compose top
```

Environment Variables

```
services:
  api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
      - DB_HOST=sqlserver
      - DB_PORT=1433
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyDb;...
```

Volumes in Compose

```
services:
  api:
    volumes:
      - ./logs:/app/logs              # Bind mount
      - apidata:/app/data             # Named volume

  sqlserver:
    volumes:
      - sqldata:/var/opt/mssql

volumes:
  apidata:
  sqldata:
```

Networks in Compose

```
services:
  api:
    networks:
      - frontend
      - backend

  db:
    networks:
      - backend

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

Restart Policies

```
services:
  api:
    restart: "no"              # Never restart
    # restart: "always"        # Always restart
    # restart: "on-failure"    # Restart on error
    # restart: "unless-stopped"# Restart unless stopped
```

Complete Example

```
version: '3.8'

services:
  api:
    build:
      context: ./MyApi
      dockerfile: Dockerfile
    container_name: myapi
    ports:
      - "8080:5000"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:5000
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyAppDB;User=sa;Passwor
        d=YourStrong@Passw0rd;TrustServerCertificate=True;
    depends_on:
      sqlserver:
        condition: service_healthy
    volumes:
      - ./MyApi/logs:/app/logs
    networks:
      - appnet

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sqlserver
    environment:
```

```
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
      - MSSQL_PID=Developer
    ports:
      - "1433:1433"
    volumes:
      - sqldata:/var/opt/mssql
    networks:
      - appnet
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P
        'YourStrong@Passw0rd' -Q 'SELECT 1'"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s

networks:
  appnet:
    driver: bridge

volumes:
  sqldata:
```

LEVEL 14: Docker Compose for .NET Project

Complete .NET Project Setup

Full docker-compose.yml for ASP.NET Core API + SQL Server

```
version: '3.8'

services:
  # ============================================
  # ASP.NET Core Web API
  # ============================================
  api:
    build:
      context: ./src/MyApi
      dockerfile: Dockerfile
    image: myapi:latest
    container_name: myapi
    ports:
      - "5000:5000"
      - "5001:5001"  # HTTPS
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:5000;https://+:5001
      - ASPNETCORE_Kestrel__Certificates__Default__Path=/app/certs/cert.pfx
      - ASPNETCORE_Kestrel__Certificates__Default__Password=password
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyAppDB;User=sa;Passwor
        d=YourStrong@Passw0rd;TrustServerCertificate=True;Encrypt=False
    depends_on:
      sqlserver:
        condition: service_healthy
    volumes:
      - ./src/MyApi/logs:/app/logs
    networks:
      - app-network

  # ============================================
  # SQL Server Database
  # ============================================
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sqlserver
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
      - MSSQL_PID=Developer
    ports:
      - "1433:1433"
    volumes:

      - sqlserver-data:/var/opt/mssql
    networks:
      - app-network
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P
        'YourStrong@Passw0rd' -Q 'SELECT 1'"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s

networks:
  app-network:
    driver: bridge

volumes:
  sqlserver-data:
```

With Redis Cache

```
version: '3.8'

services:
  api:
    build: .
    ports:
      - "5000:5000"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyAppDB;User=sa;Passwor
```

```
        d=YourStrong@Passw0rd;TrustServerCertificate=True;
      - Redis__Host=redis
      - Redis__Port=6379
    depends_on:
      - sqlserver
      - redis
    networks:
      - app-network

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
    ports:
      - "1433:1433"
    volumes:
      - sqlserver-data:/var/opt/mssql
    networks:
      - app-network

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    networks:
      - app-network
    command: redis-server --appendonly yes

networks:
  app-network:

    driver: bridge

volumes:
  sqlserver-data:
  redis-data:
```

With Nginx Reverse Proxy

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - api
    networks:
      - app-network

  api:
    build: .
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

Development vs Production Override

docker-compose.yml (base):

```
version: '3.8'

services:
  api:
    build: .
    ports:
      - "5000:5000"
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
```

```
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
```

docker-compose.override.yml (development):

```
version: '3.8'

services:
  api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
    volumes:
      - ./src:/app/src    # Hot reload enabled
    command: dotnet watch run

  sqlserver:
    ports:
      - "1433:1433"
```

docker-compose.prod.yml (production):

```
version: '3.8'

services:
  api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    restart: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 3s
      retries: 3
    deploy:
      resources:
        limits:
          memory: 512M

  sqlserver:
    environment:
      - MSSQL_PID=Standard
    restart: always
```

Run with override:

```
# Development (uses .override.yml automatically)
docker compose up -d

# Production
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Running EF Core Migrations

```
# Run migrations
docker compose exec api dotnet ef database update

# Or in code (Program.cs)
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
}
```

LEVEL 15: Environment Variables and Configuration

Complete Configuration Guide

Environment Variable Basics

```
# Set single variable
docker run -e ASPNETCORE_ENVIRONMENT=Production myapi

# Set multiple variables
docker run \
  -e ASPNETCORE_ENVIRONMENT=Production \
  -e DB_HOST=sqlserver \
  -e API_KEY=secret123 \
  myapi

# From .env file
docker run --env-file .env myapi

# .env file content:
# ASPNETCORE_ENVIRONMENT=Production
# DB_HOST=sqlserver
# API_KEY=secret123
```

Environment Variables in Dockerfile

```
# Set at build time
ENV ASPNETCORE_ENVIRONMENT=Production
ENV APP_NAME=MyApi
ENV APP_VERSION=1.0.0

# Can override at runtime
docker run -e ASPNETCORE_ENVIRONMENT=Development myapi
# This overrides the ENV in Dockerfile
```

Environment Variables in Docker Compose

```
services:
  api:
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
      - ASPNETCORE_URLS=http://+:5000
      - ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyDb;...

# Or dictionary format
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_URLS: http://+:5000

# From env_file
    env_file:
      - ./config.env

# Multiple env_files
    env_file:
      - ./common.env
      - ./prod.env
```

.NET Configuration Hierarchy

```
// Priority (highest to lowest):
// 1. Environment variables
// 2. Command-line arguments
// 3. appsettings.{Environment}.json
// 4. appsettings.json
// 5. Default values

// Connection string in different formats:

// appsettings.json
"ConnectionStrings": {
  "DefaultConnection": "Server=localhost;..."
}

// Environment variable (replace : with __)
ConnectionStrings__DefaultConnection=Server=sqlserver;...

// .NET reads this automatically!
```

Environment-Specific Settings

appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;..."
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

appsettings.Development.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;..."
  },
  "Logging": {
    "LogLevel": {
      "Default": "Debug"
    }
  }
}
```

appsettings.Production.json:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Warning"
    }
  }
}
```

Secrets Management

Development:

```
# .env file (never commit to Git!)
# .gitignore should include .env
ASPNETCORE_ENVIRONMENT=Development
DB_PASSWORD=YourStrong@Passw0rd
```

Production:

```
# Use Docker secrets or external secrets manager
# Docker secrets (Swarm mode)
echo "secret_password" | docker secret create db_password -

# AWS Secrets Manager
# Azure Key Vault
# HashiCorp Vault
```

Best Practice:

```
# docker-compose.yml - Don't put real secrets here!
# Use placeholders or reference external secrets
services:
  api:
    environment:
      - ConnectionStrings__DefaultConnection=Server=${DB_HOST};...
```

Configuration Best Practices

```
#  Do: Use environment variables for configuration
-e ASPNETCORE_ENVIRONMENT=Production

#  Do: Use secrets management in production
# AWS Secrets Manager, Azure Key Vault

#  Don't: Hardcode passwords
# "password123" in code

#  Don't: Commit secrets to Git
# Add .env to .gitignore

#  Do: Use different settings per environment
```

```
# Development: Verbose logging, debug mode
# Production: Minimal logging, no debug
```

LEVEL 16-30: Advanced Topics

This section continues with:

- Debugging and Troubleshooting
- Docker Cleanup
- Advanced Dockerfile
- Docker Registry
- Docker Security
- Resource Management
- Real Project Setup
- EF Core with Docker
- Microservices
- CI/CD
- AWS Deployment
- Kubernetes Introduction

Due to length, these advanced levels are covered in the comprehensive markdown files created.